



FUNCTIONAL DOCUMENTATION & SOFTWARE REQUIREMENTS SPECIFICATION

TechpioAsset

Enterprise Office & IT Asset Management Platform

Manage Assets. Control Costs. Simplify Operations.

Document type	Functional Documentation & SRS (baseline for workflow redesign)
Product	TechpioAsset — multi-tenant SaaS (Web + Mobile + REST API)
Version	1.0 — as-built baseline
Status	Reverse-engineered from source; assumptions marked inline
Primary source	Monorepo at <code>E:\TechpioAsset</code> (apps: api, web, mobile; packages: domain, contracts, ui-tokens, config)
Live deployment	https://piotask.com (self-hosted Docker, single VPS)
Intended audience	Developers, UI/UX designers, QA engineers, business analysts, stakeholders

Confidential. This specification documents the application exactly as implemented in source, with clearly marked assumptions where behaviour was inferred, and prioritised recommendations for the planned workflow redesign.

Table of Contents

1	Project Overview
2	System Architecture
3	User Roles & Responsibilities
4	Module Documentation
5	Screen-by-Screen Documentation (Web & Mobile)
6	Field Documentation (Forms)
7	Complete End-to-End Workflow
8	Approval Workflow
9	Status Transition Documentation
10	Notification Flow
11	Business Rules
12	Database Overview
13	API Documentation
14	Workflow Diagrams & Flowcharts
15	UX / UI Improvement Suggestions
16	Missing Features & Enhancement Recommendations
17	Development Best Practices
18	Future Roadmap
19	Appendix & Glossary

READING GUIDE Sections 1–14 describe the system **as it is today** (verified against source, with **ASSUMPTION** and **GAP** flags). Sections 15–18 are the **redesign recommendations**. Every non-obvious claim was traced to a source file during analysis.

1 Project Overview

TechpioAsset is a multi-tenant SaaS platform that lets an organisation track every physical and IT asset it owns — from procurement and assignment through maintenance, transfer, and disposal — while controlling cost, enforcing approvals, and preserving a complete audit trail. It is delivered as a responsive web application, a native mobile app (Android/iOS via Expo), and a shared REST API.

1.1 Purpose & Problem Solved

Organisations lose money and control when asset data lives in spreadsheets: unknown who-has-what, missed warranties, unverified vendor invoices, uncontrolled spend, and no accountability. TechpioAsset centralises the entire asset lifecycle behind a permission-driven workflow so that:

- every asset has a single record with status, location, owner, cost (permission-gated), warranty, and full history;
- employees self-serve requests and confirm receipts; managers and departments approve through a configurable chain;
- Finance captures and *deterministically* verifies invoices (AI may assist extraction but never decides);
- IT/Office admins run maintenance and drive lifecycle transitions through validated state machines;
- every sensitive action is written to an append-only audit log.

1.2 Scope of the Product

Domain	Capability (in scope, implemented)
Asset lifecycle	Create, edit, import (Excel), assign/return, transfer, status machine (18 states), condition history, QR labelling, disposal (as a record, never a hard delete), depreciation (computed on read).
Requests & approvals	13 request types, multi-step configurable approval chains, cost-threshold skipping, line-manager routing, fulfilment pipeline, cancellation, comments.
Procurement & invoices	Manual entry or document upload, AI-assisted extraction (opt-in), deterministic verification engine, human verify/reject, vendor & purchase-order references, invoice-line-to-asset linking.
Maintenance	Repair/inspection/warranty-claim records, schedule → start → complete lifecycle, service cost + downtime capture, asset out-of-service cascade, repair-vs-replace guidance.
People & lifecycle	User directory, role/status management, onboarding & offboarding task plans with asset-recovery gating.
Governance	RBAC (8 system roles, ~55 permissions, 4 data scopes), append-only audit (33 action types), reporting (7 report types, CSV/XLSX), notifications (in-app/email/push), AI governance (off by default, human-review mandated).
Platform	Web SPA, native mobile (QR scan, offline stocktake, biometric unlock, push), provider abstractions for storage/AI/mail/queue/cache/SSO so the deployment target is configurable.

1.3 Primary Personas

Persona	What they do in TechpioAsset
Employee	Views their assigned equipment, confirms receipt, raises requests, reports damage/repair needs.
Manager	Approves their direct reports' requests; sees assets held by their team.
IT Admin / Office Admin	Own the asset estate: create/assign/transfer assets, run maintenance, fulfil on/offboarding, adjust inventory.
HR	Manages employees and on/offboarding; raises requests on behalf of staff.
Finance	Owns cost visibility, captures & verifies invoices, manages vendors/POs, sets asset prices (write-once).
Auditor	Read-only oversight across the whole company (no cost/financial visibility).
Super Admin	Full control: configures roles, workflows, settings, AI; the only role that can correct a locked price.

1.4 Design Philosophy (as observed in source)

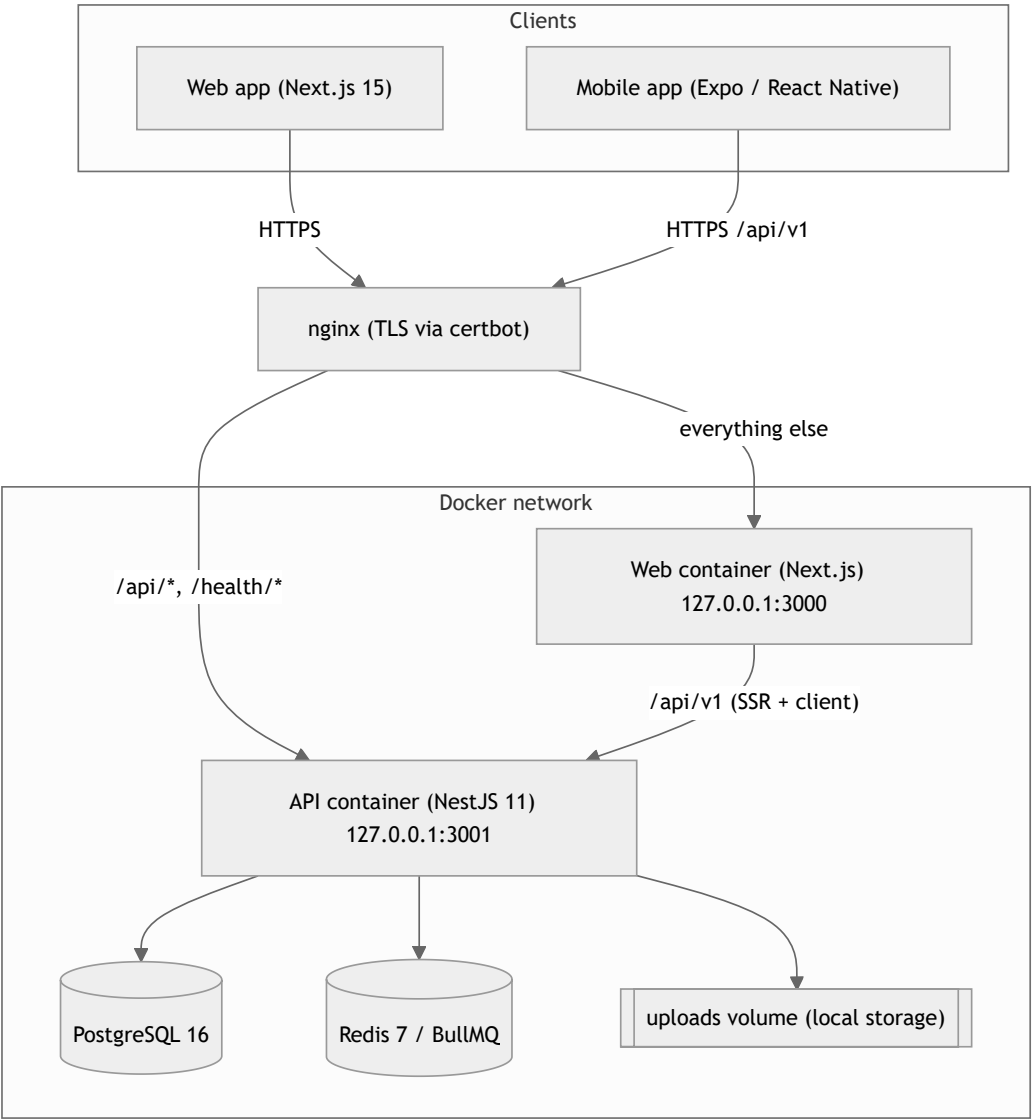
- **Permissions, not role names.** No code branches on a role name; guards check permission strings so roles are reconfigurable at runtime by a Super Admin.

- **Deny by default.** Every API route is authenticated unless explicitly public; authority is re-resolved from the database on every request, never trusted from the token.
- **Deterministic money, assisted by AI — never decided by AI.** All financial math and invoice verification are pure, testable functions; AI can suggest, a human always decides.
- **Immutability & history.** Financial, assignment, and audit rows have no delete path; transfers and disposals are recorded events, not overwrites.
- **State machines everywhere.** Asset, request, invoice, and maintenance transitions are guarded by declarative state machines that reject illegal moves.

2 System Architecture

2.1 High-Level Topology

TechpioAsset is a pnpm/Turborepo monorepo. Three applications (API, Web, Mobile) share four packages (domain logic, API contracts, design tokens, tooling config). In production every service runs in Docker on a single VPS behind the host's nginx, which terminates TLS and reverse-proxies to localhost-bound app containers; PostgreSQL and Redis stay private to the Docker network.



2.2 Monorepo Layout

Path	Type	Responsibility
<code>apps/api</code>	NestJS 11 service	REST API, business logic, Prisma/Postgres, Redis/BullMQ, provider abstractions.
<code>apps/web</code>	Next.js 15 / React 19	Web SPA (App Router, TanStack Query, react-hook-form + Zod, Radix UI, Tailwind 4, Recharts).
<code>apps/mobile</code>	Expo ~52 / RN 0.76	Native app: QR scan, offline stocktake, biometric unlock, push, camera bill capture.
<code>packages/domain</code>	Pure TS library	Permission catalogue, state machines, workflow rules, AI-gate, money/depreciation logic, typed errors. No I/O.
<code>packages/contracts</code>	Zod + types	Request/response schemas shared by API and clients; response envelope, <code>ProblemDetails</code> , <code>error→status</code> map.

<code>packages/ui-tokens</code>	Design tokens	Status/tone colour tokens used identically by web and mobile.
<code>packages/config</code>	Tooling	Shared eslint/tsconfig/prettier bases.

2.3 API Conventions

- **Global prefix** `/api/v1` on every route (health probes `/health/live`, `/health/ready` are excluded).
- **Guard chain:** `ThrottlerGuard` → `JwtAuthGuard` → `PermissionsGuard`. Rate-limiting runs before authentication so unauthenticated floods are cheap to reject.
- **Success envelope:** `{ data, meta: { requestId, timestamp } }`; list endpoints add `meta.page`. File downloads and redirects pass through unwrapped.
- **Error model:** RFC 9457 `application/problem+json` from a single global filter — `{ type, title, status, code, requestId, timestamp, detail?, errors?[] }`. Internal detail (stacks, SQL) is logged, never returned.
- **Pagination contract:** `page` (1), `pageSize` (25, max 100), `sort`, `order` (desc), `q` (free-text, ≤200 chars); ordering uses a column whitelist.

2.4 Provider Abstraction Pattern

Each external capability is an abstract class used as the DI token; a `@Global()` module selects the concrete implementation from an environment variable at boot. Real providers fail fast (throw in their constructor) if selected without credentials. This makes the deployment target fully configurable without touching application code.

Capability	Implementations	Selector env	Default
Storage	Local · Azure Blob · S3	<code>STORAGE_PROVIDER</code>	local
AI document extraction	Mock · Azure Doc Intelligence · Anthropic (Claude)	<code>AI_PROVIDER</code>	mock
Mail	Local (.eml file) · SMTP	<code>MAIL_PROVIDER</code>	mock
Job queue	In-process · BullMQ (Redis)	<code>QUEUE_PROVIDER</code>	in-process
Cache	In-memory · Redis	<code>CACHE_PROVIDER</code>	memory
Rate-limit store	In-process · Redis	<code>RATE_LIMIT_STORAGE</code>	memory
Chat (Teams/Slack)	Mock · Webhook	<code>CHAT_PROVIDER</code>	mock
Push	Mock · Expo	<code>PUSH_PROVIDER</code>	mock
SSO	Disabled · Microsoft Entra (OIDC)	auto when Entra vars present	disabled

2.5 Security & Cross-Cutting Infrastructure

Concern	Implementation
Access token	JWT (HS256), TTL 15m, held in client memory. Used only to identify the subject — roles/permissions are re-resolved from the DB every request.
Refresh token	Opaque 48-byte random, only SHA-256 hash stored; httpOnly cookie (web) / <code>X-Refresh-Token</code> header (mobile); rotating with family-wide reuse detection.
Passwords	argon2id (OWASP params: 19 MiB, 2 iterations, 1 lane); ≥12 chars with upper/lower/digit.
MFA	TOTP (6-digit/30s); shared secret encrypted at rest with AES-256-GCM, never returned by any endpoint.
Brute-force	Login throttled 10/min per IP + per-account lockout (5 attempts → 15 min).
Headers / CORS	helmet with strict CSP (<code>default-src 'none'</code>), explicit CORS origin allowlist (never wildcard), <code>credentials: true</code> .
Logging	Deep redaction of any secret-looking key + bearer/JWT strings in free text; applied to all error-log context.
Tenancy	Prisma client extension injects <code>companyId</code> on every query; scope filters further restrict by role scope in the repository layer.
Uploads	Magic-byte signature validation (PDF/JPEG/PNG/HEIC), size caps, SHA-256 dedup; documents served only via signed URLs. Malware scan is a hook (status defaults to <code>SKIPPED</code> , not silently <code>CLEAN</code>).

2.6 Technology Stack

Layer	Technology	Version
Monorepo / build	Turborepo · pnpm	2.3 · 11.15
Language / runtime	TypeScript · Node.js	5.7 · ≥20.11 (built on Node 22)
API	NestJS (Express 5) · Prisma	11 · 6.2
Database / cache	PostgreSQL · Redis · BullMQ	16 · 7 · 5.80
Validation / money	Zod · decimal.js	3.24 · 10.4
Auth	@nestjs/jwt · jose · @node-rs/argon2 · otpauth	—
Web	Next.js · React · TanStack Query · Radix · Tailwind · Recharts · react-hook-form	15 · 19 · 5 · — · 4 · 3 · 7
Mobile	Expo · React Native · expo-router/camera/secure-store/sqlite	52 · 0.76
AI / mail / files	@anthropic-ai/sdk · nodemailer · exceljs · multer	—
Testing	Vitest (+ unplugin-swc, supertest)	2.1

3 User Roles & Responsibilities

Authorisation has two orthogonal axes: **permissions** (what actions you may perform) and **data scope** (which records you may see). Roles are named bundles of permissions; scope is derived separately. A user may hold several roles — permissions are the *union*, and the *widest* scope wins. The eight roles below are seeded system roles; a Super Admin may create custom roles (custom roles resolve to **OWN** scope by default).

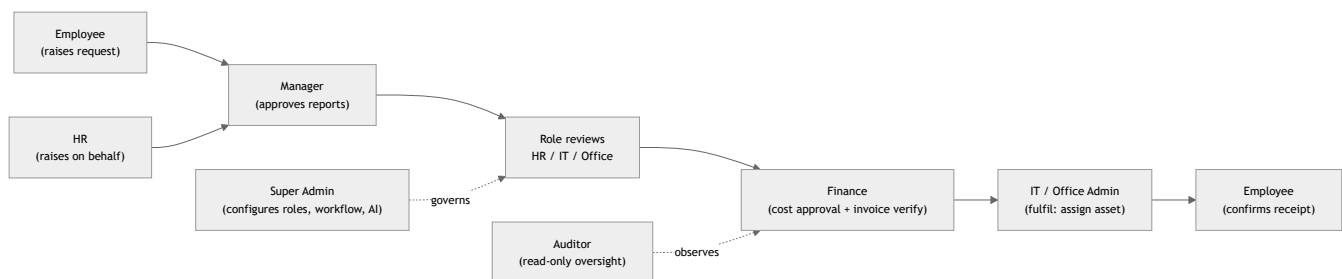
3.1 System Roles

Role	Responsibility	Default scope	Read-only
Super Admin	Full control incl. role/permission/workflow/settings/AI configuration and price corrections.	ALL	No
IT Admin	Asset lifecycle, assignments, transfers, maintenance, inventory; read-only on invoices/cost.	ALL	No
HR	Employees, onboarding/offboarding; raises requests on behalf of staff. No financial visibility by default.	ALL	No
Office Admin	Office assets, inventory, maintenance; fulfils on/offboarding. No cost visibility.	ALL	No
Finance	Cost visibility, invoice capture & verification, vendors/POs, asset pricing (write-once).	ALL	No
Manager	Approves their direct reports' requests; sees assets held by their team.	DIRECT_REPORTS	No
Employee	Requests assets, views/acknowledges own assets, reports maintenance.	OWN	No (minimal writes)
Auditor	Read-only oversight company-wide. Can never be granted a write permission (structural invariant).	ALL	Yes

3.2 Data Scopes

Scope	Meaning	Fail-safe note
ALL	Every record in the company (tenant-filtered only).	—
DEPARTMENT	Records for the user's department.	A departmentless user sees nothing , not everything (fail-closed).
DIRECT_REPORTS	Self + the user's direct reports (managers).	Line-manager approval matches the requester's <i>actual</i> manager, not any manager-role holder.
OWN	Only the user's own records.	Employees additionally see requests raised on their behalf by HR.

3.3 Role Interaction (typical request & asset flow)



3.4 Role × Key-Permission Matrix

✓ = granted. Super Admin holds every permission. Sourced from the **ROLE_PERMISSIONS** matrix.

Permission	Super	IT	HR	Office	Fin	Mgr	Emp	Aud
assets:read	✓	✓	✓	✓	✓	✓	✓	✓
assets:create / update / assign	✓	✓		✓				

assets:cost:read (see & set price)	✓				✓			
inventory:read / adjust	✓	✓		✓	read			read
invoices:read	✓	✓			✓			✓
invoices:upload / verify	✓				✓			
requests:create	✓	✓	✓	✓	✓	✓	✓	
requests:approve	✓	✓	✓	✓	✓	✓		
requests:cancel	✓						✓	
maintenance:read	✓	✓		✓	✓			✓
maintenance:manage	✓	✓		✓				
maintenance:request	✓						✓	
employees:read / manage	✓	read	✓	read		read		read
onboarding / offboarding	✓	fulfil	manage	fulfil				
reports:read	✓	✓	✓	✓	✓	✓		✓
audit:read	✓	✓			✓			✓
users:read / roles:manage	✓							read
ai:configure	✓							
ai:review-results	✓				✓			
workflows:configure / settings:manage	✓							

DELIBERATE GAPS HR has *no* cost or invoice visibility by default; IT Admin can read invoices but not upload them; Auditor has no cost visibility and no financial reports; **ai:configure** is Super-Admin only.

3.5 Per-Role Deep Dive

Super Admin

Holds all permissions. Unique powers: role/permission/workflow/settings/AI configuration, and the **only** role that can correct a write-once asset price (via **permissions:manage**, audit-logged as **ASSET_COST_CHANGED**). Restrictions: cannot self-suspend; the last active Super Admin cannot be removed or demoted.

IT Admin

Owns the technical asset estate: create/update/import assets, assign/return/transfer, inventory read+adjust, maintenance read+manage, invoices read-only, requests create/read/approve (IT review step), on/offboarding fulfil, QR generate/print. No cost visibility, no invoice upload/verify, no asset-disposal grant.

HR

Manages employees (read/create/import), onboarding/offboarding *plans*, and can raise requests **on behalf of** staff (**requests:create-on-behalf**). Approver at the HR review step. Explicitly no financial/cost visibility unless individually granted.

Office Admin

Like IT for office/furniture/kitchen assets (create/update/assign/return/transfer — no import), inventory adjust, maintenance manage, requests approve (Office review step), on/offboarding fulfil. No cost visibility, no invoice capture, no audit read.

Finance

Owns money: cost visibility (**assets:cost:read**), invoice upload/correct/verify, vendors manage, POs manage, requests approve (Finance step), **ai:review-results**, audit read. The only role besides Super Admin that can **set** an asset price — but cannot *edit* it afterwards (write-once). Cannot run repairs (maintenance:manage stays with IT/Office).

Manager

Assets read, requests create/read/approve, employees read, reports read. Scope DIRECT_REPORTS: sees assets held by self or direct reports; approves only requests where they are the requester's actual line manager (a `LINE_MANAGER` step is satisfied by the requester's manager, not any manager-role holder).

Employee

Assets read (own), requests create/read/cancel, maintenance request. Scope OWN: sees only assets assigned to them and their own requests (plus requests raised for them by HR). No approval authority; internal request comments are hidden from them.

Auditor

Read-only across assets, inventory, invoices, vendors, POs, requests, employees, maintenance, reports (read+export), users, audit.

Structural invariant: a read-only role can never be handed a non-read permission — enforced in the domain layer (`assertGrantAllowed`), on the `Role.isReadOnly` flag, and re-checked at seed time. No cost visibility or financial reports.

3.6 Enforcement Mechanics

- **Authentication is global-deny.** Every route is protected unless marked public; the JWT identifies the subject only — roles/permissions are re-resolved from the database on each request via `UserRole → Role → RolePermission → Permission`.
- **Permission checks** use `@RequirePermissions(...)`; multiple decorators are AND-ed; missing keys are logged, never returned to the caller.
- **Scope filtering** lives in the repository layer (so reports and exports — the endpoints most likely to leak — are covered), composing a tenant filter first, then a role-scope filter.
- **Cost visibility** is column-level: `redactCost` strips monetary fields for actors without `assets:cost:read` — it removes the columns, not the row, so HR can still manage an asset it may not price.
- **Write-once price:** setting a price needs cost permission; changing it afterwards is refused for everyone except a Super Admin, and every change is audited.

ASSUMPTION / GAP The runtime role-mutation API (create role / grant permission at runtime) is **not yet implemented**; `assertGrantAllowed` currently guards only the seed path. The domain layer is ready for it — when exposed, the runtime endpoint must call the same guard. Custom (non-system) roles work for permissions but resolve to `OWN` scope until scope-for-custom-roles is wired.

4 Module Documentation

The application is organised into functional modules, each backed by an API feature module, one or more screens on web and mobile, and a permission set. UI gating is convenience-only — every route and endpoint is independently enforced server-side.

Module	Purpose	Primary permissions	Key dependencies
Access & Identity	Login, MFA, SSO, refresh/rotation, password & email flows, current-user resolution.	public + authenticated	argon2, JWT, otpauth, Entra OIDC
Dashboard	KPI tiles, charts, warranty timeline, recommendations, quick actions.	none (self-scoped); spend needs <code>assets:cost:read</code>	Assets, Reports
Assets	Full asset lifecycle: CRUD, import, assign/return, transfer, status machine, QR, price capture, condition history.	<code>assets:read/create/update/import/assign/return/transfer/cost:read</code>	Categories, Offices, Vendors, Users
Requests & Approvals	Raise, submit, approve/reject, fulfil, cancel, comment on asset requests.	<code>requests:create/read/approve/cancel/create-on-behalf</code>	Workflow engine, Users, Assets, Notifications
Invoices & Procurement	Capture (upload/manual), AI extraction, deterministic verification, human decision, line→asset linking.	<code>invoices:read/upload/correct-extraction/verify</code> , <code>vendors:*</code>	Storage, AI provider, Vendors, Assets
Maintenance	Repair/inspection records, schedule→start→complete lifecycle, cost/downtime, repair-vs-replace advice.	<code>maintenance:read/request/manage</code>	Assets (out-of-service cascade), Vendors
People & Lifecycle	User directory, role/status management, onboarding/offboarding task plans.	<code>employees:read</code> , <code>users:manage</code> , <code>roles:manage</code> , <code>onboarding/offboarding:*</code>	Assets (recovery gating), Notifications
Inventory	Quantity-tracked stock: balances, movements, offline mobile stocktake.	<code>inventory:read/adjust</code>	Mobile sync, Categories
Reports	7 report types (inventory, spend×3, depreciation, warranty, maintenance), CSV/XLSX export, scheduled reports.	<code>reports:read/export</code> (+ <code>assets:cost:read</code> for financial)	Assets, Invoices, Maintenance
Audit	Append-only trail of 33 sensitive action types with before/after diffs.	<code>audit:read</code>	all modules (writers)
Notifications	In-app centre, email/push fan-out, per-user preferences, mandatory types.	authenticated (self-scoped)	Queue, Mail, Push providers
AI Settings	Company AI config: global on/off, human-review requirement, per-feature modes.	<code>ai:configure</code>	AI provider, AI-gate domain logic
Governance / Config	Roles, workflows, categories, settings; scheduled jobs; org reference data.	<code>*:manage</code> , <code>settings:manage</code>	seed data, cron engine

5 Screen-by-Screen Documentation

5.1 Web Navigation Map

The authenticated web app wraps every page in an App Shell (header with global asset search, sidebar, notification bell, profile menu). Sidebar items are hidden unless the permission is held.

#	Sidebar item	Route	Permission gate
1	Dashboard	/dashboard	always
2	Assets	/assets	assets:read
3	My assets	/my-assets	always (self)
4	Requests	/requests	always
5	Invoices	/invoices	invoices:read
6	People	/people	employees:read
7	Maintenance	/maintenance	maintenance:read
8	Reports	/reports	reports:read
9	Audit log	/audit	audit:read
10	AI settings	/settings/ai	ai:configure

5.2 Web Screens

Screen	Purpose	Key actions	Main API calls
Marketing (Home / About / Contact)	Public landing, company info, enquiry form (mailto).	Sign in, contact	none
Login	Email/password + optional TOTP MFA + Entra SSO; Android APK download link.	Sign in, Verify, Continue with Microsoft	POST /auth/login, GET /auth/sso/available
Dashboard	6 KPI tiles, spend breakdown (cost-gated), donut/area/gauge/pie/bar charts, warranty timeline, rule-based recommendations, quick actions, attention lists.	Add asset, quick links	GET /assets?pageSize=100, GET /reports?type=SPENDING_BY_CATEGORY
Assets list	Paged table (25), search + status filter, bulk status change (confirm on destructive), CSV export, Excel import.	Export, Import, Add, Bulk status	GET /assets, POST /assets/bulk/status
Asset detail	Full record; one-time price capture (locks after save); condition/assignment history.	Edit, Set price	GET /assets/:id, PATCH /assets/:id/price
Add / Edit asset	Create (status defaults AVAILABLE) / edit with optimistic-concurrency version; price field only when cost-permitted.	Create / Save changes	POST /assets, PATCH /assets/:id
Import assets	Excel bulk import matched by serial (idempotent); auto-creates named employees; per-row error report.	Upload .xlsx	POST /assets/import
My assets	Card grid of equipment issued to the current user.	Open asset	GET /assets?assignedUserId=...
Requests list	Paged table; debounced search, status/type filters, All / Awaiting-me toggle (approvers), CSV export.	New request, Export	GET /requests?awaitingMe&q&status&type
New request	Type, priority, business reason (≥ 10), required-by, items field-array; created draft then submitted.	Submit for approval	POST /requests, POST /requests/:id/submit
Request detail	Business reason, items, decision card (approve/reject on current step), comments, approval-chain sidebar.	Approve / Reject, Comment	GET /requests/:id, POST /requests/:id/decision
Invoices list	Paged ledger; verification-status pills; scan-a-bill.	Scan a bill	GET /invoices
Invoice review	Split-screen: document preview vs extracted fields (low-confidence highlighted), line items, deterministic issues, human verify/reject.	Verify / Reject	GET /invoices/:id, GET /storage/preview/:doc, POST /invoices/:id/decision

Invoice upload	File (PDF/JPG/PNG/HEIC) + optional vendor; AI extraction or manual entry.	Upload and process	GET /vendors, POST /invoices/upload
Maintenance list / detail	Estate maintenance list; detail with start-work / complete-and-return actions (cost, downtime, notes).	Start / Complete	GET /maintenance, POST /maintenance/:id/{start,complete}
People	User directory; Manage modal edits roles (roles:manage) and status (users:manage); cannot change own status.	Manage roles/status, Export	GET /users, PATCH /users/:id/{roles,status}
Reports	7 report types (financial ones cost-gated); dynamic columns; CSV/XLSX export.	Run, Export	GET /reports?type&format
Audit log	Append-only trail (30/pg); action/entity filters; compact field-diff details.	Filter	GET /audit?action&entityType
AI settings	Global on/off, require-human-review, 12 per-feature modes; provider disclosure.	Save	GET /ai-config, PATCH /ai-config
Profile	Read-only identity + "what you can see" scope; 2FA link.	—	none (uses session)

STUB ROUTES The profile menu / header link to [/my-requests](#) , [/settings/notifications](#) , [/settings/appearance](#) , [/settings/security](#) , and [/help](#) , but no page exists for these yet — dead links to implement or remove.

5.3 Mobile Navigation Map

The mobile app uses a five-item bottom tab bar plus a grouped "More" menu; items are hidden without the permission. Records screens are read-mostly and point to the web app for management.

Bottom tab	Route	Gate
Home	(tabs)/index	none
Assets	(tabs)/assets	assets:read
Requests	(tabs)/requests	none
Approvals	(tabs)/approvals	requests:approve
More	(tabs)/more	none

More menu — *Capture*: Scan a code (assets:read), Capture bill (invoices:upload), Inventory count (inventory:adjust). *Records*: Invoices (invoices:read), Maintenance (maintenance:read), People (users:read), Reports (reports:read), Audit log (audit:read). *Account*: AI settings (ai:configure), Profile, Sign out.

5.4 Mobile Screens (highlights)

Screen	Purpose	Key API
Home	Greeting, stat cards (my assets / my requests / to-approve), my-assets list.	GET /assets?assignedUserId, /requests, /requests?awaitingMe
Assets	All-company assets with client search → asset detail.	GET /assets?pageSize=200
Requests	Own requests + create (single-item form, reason ≥10) with validation/success feedback.	GET/POST /requests, POST /requests/:id/submit
Approvals	Approver inbox; reloads on focus; tap to review/decide.	GET /requests?awaitingMe=true
Scan	QR/barcode → resolve to authorised asset.	GET /assets/by-qr:code
Capture bill	Photograph an invoice → multipart upload → extraction outcome.	POST /invoices/upload
Inventory	Offline stocktake: SQLite queue of scans, idempotent batch sync.	POST /mobile/inventory/sessions, POST /mobile/sync
Asset / Request detail	Confirm receipt / report damage; approve/reject with comment.	POST /assets/assignments/:id/acknowledge, POST /requests/:id/decision
Invoices / Maintenance / People / Audit / Reports / AI settings	Read-only lists/summaries; management stays on web.	GET /invoices, /maintenance, /users, /audit, /ai-config

5.5 Web vs Mobile Parity

Capability	Web	Mobile
Add/Edit asset, Excel import	✓	—
Invoice verify (split-screen), Maintenance manage, People role/status, editable AI settings	✓	read-only
Reports export, in-app notification centre, SSO, marketing site	✓	—
QR/barcode scan, offline stocktake, camera bill capture, biometric unlock, push registration	—	✓
Requests, Approvals, Asset detail, Dashboard/Home, Profile	✓	✓

DESIGN INTENT Mobile is deliberately field-oriented: capture (scan/photo/stocktake) and act (approve, confirm receipt, report damage) in the field; heavy management and configuration remain on the web.

6 Field Documentation (Forms)

All create/update payloads validate against Zod schemas in `packages/contracts` (there are no class-validator DTOs). Money is a shared string regex `^\d{1,12}(\.\d{1,2})?$/` — non-negative, ≤ 12 integer digits, ≤ 2 decimals. Currency is always a 3-letter uppercase code (no ISO-4217 membership check). IDs are plain strings; existence/tenancy is checked in the service layer, not the schema.

6.1 Create Asset

Contract `createAssetSchema`; web form `assets/new`. Price field renders only for actors with `assets:cost:read`. On the web, status is hardcoded `AVAILABLE` on submit.

Field	DB column	Type	Req	Validation	Default	Editable by
name	Asset.name	string	Yes	trim, 1–200	—	assets:create/update
assetTag	Asset.assetTag	string	Yes	trim, 1–64; unique per company	—	assets:create/update
categoryId	Asset.categoryId	string (FK)	Yes	min 1; from GET /categories	—	assets:create/update
subcategoryId	Asset.subcategoryId	string (FK)	No	dependent on category	—	assets:create/update
trackingType	Asset.trackingType	enum	No	INDIVIDUAL QUANTITY	INDIVIDUAL	assets:create/update
brand / model / serialNumber	Asset.*	string	No	trim, ≤ 120 each; serial unique per company (documented-exception override)	—	assets:create/update
barcode / manufacturerPartNumber / purchaseOrderNumber	Asset.*	string	No	trim, ≤ 120 / ≤ 64	—	assets:create/update
purchaseDate / warrantyStartDate / warrantyEndDate / expectedReplacementDate	Asset.*	date	No	coerced date	—	assets:create/update
purchaseCost	Asset.purchaseCost	money string	No	money regex; write-once after set	—	assets:cost:read (set); Super Admin (correct)
currency	Asset.currency	string(3)	No	length 3, uppercased	—	assets:cost:read
vendorId / officeId / buildingId / floorId / roomId / departmentId	Asset.*	string (FK)	No	reference lookups	—	assets:create/update
condition	Asset.condition	enum	No	NEW GOOD FAIR POOR DAMAGED UNUSABLE	GOOD	assets:create/update
status	Asset.status	enum(18)	No	ASSET_STATUSES	DRAFT (API) / AVAILABLE (web)	assets:create/update
notes / description	Asset.*	string	No	trim, ≤ 4000 / ≤ 2000	—	assets:create/update
duplicateExceptionReason	(not persisted as column)	string	No	trim, 10–500 — unlocks a duplicate-serial create	—	assets:create

6.2 Edit Asset

`updateAssetSchema` = create schema `.partial()` (all fields optional) plus optimistic-lock `version` (stale $\rightarrow$ 409). Web edit form adds a **Status** select and omits Price (managed via the dedicated price endpoint).

6.3 Create Request

Contract `createRequestSchema`; web form `requests/new` (react-hook-form field array). Mobile sends a simplified single-item variant.

Field	DB column	Type	Req	Validation	Default
type	AssetRequest.type	enum(13)	Yes	REQUEST_TYPES (onboarding, replacement, damage, loss, upgrade, temporary, project, office, kitchen, accessibility, additional-equipment, repair, return)	—
priority	AssetRequest.priority	enum	No	LOW NORMAL HIGH URGENT	NORMAL
businessReason	AssetRequest.businessReason	string	Yes	trim, 10–2000 ("approvers read this first")	—
beneficiaryId	AssetRequest.beneficiaryId	string (FK)	No	on-behalf needs requests:create-on-behalf	self
requiredBy	AssetRequest.requiredBy	date	No	coerced date	—
estimatedCost / currency	AssetRequest.*	money / string(3)	No	money regex / len 3	—
items[]	RequestItem[]	array	Yes	1–50 items	1 empty row
• item.description	RequestItem.description	string	Yes	trim, 1–500	—
• item.quantity	RequestItem.quantity	number	Yes	int, 1–9999	1
• item.estimatedCost / preferredSpec	RequestItem.*	money / string	No	money regex / ≤1000	—

6.4 Invoice — Create & Upload

Manual create (**createInvoiceSchema**): vendorId, invoiceNumber (1–100, unique per company), invoiceDate, currency, subtotal, total (money, required); discount/tax/shipping/otherCharges (money, default 0.00); paymentStatus (default UNPAID); lines[] (1–200, each: lineNumber int>0, description 1–500, quantity, unitPrice, lineTotal money, optional serialNumbers[], warrantyMonths 0–600). **Total formula is fixed**: subtotal – discount + tax + shipping + otherCharges; discount may not exceed subtotal.

Document upload (**POST /invoices/upload**): no Zod body — imperative validation only. Multer + ParseFilePipe cap the file at 30 MB; byte-level magic-number check accepts PDF/JPEG/PNG/HEIC; SHA-256 dedup; scan status defaults SKIPPED. Body **vendorId** / **invoiceNumber** are optional unvalidated strings.

6.5 Maintenance — Create & Complete

Create (**createMaintenanceSchema**): assetId (req), type (SCHEDULED|REPAIR|INSPECTION|WARRANTY_CLAIM|CALIBRATION|CLEANING), title (1–200), description (≤2000), vendorId, isInternal (default false), scheduledFor. **Complete**: serviceCost (money), currency, downtimeHours (**^d{1,6}(\.d{1,2})?\$**), resolutionNotes (≤2000), replacementRecommended (default false), restoreAsset (default true). Note: there is currently no web *create-maintenance* form — only a "Complete work" form on the detail screen; the create endpoint exists server-side.

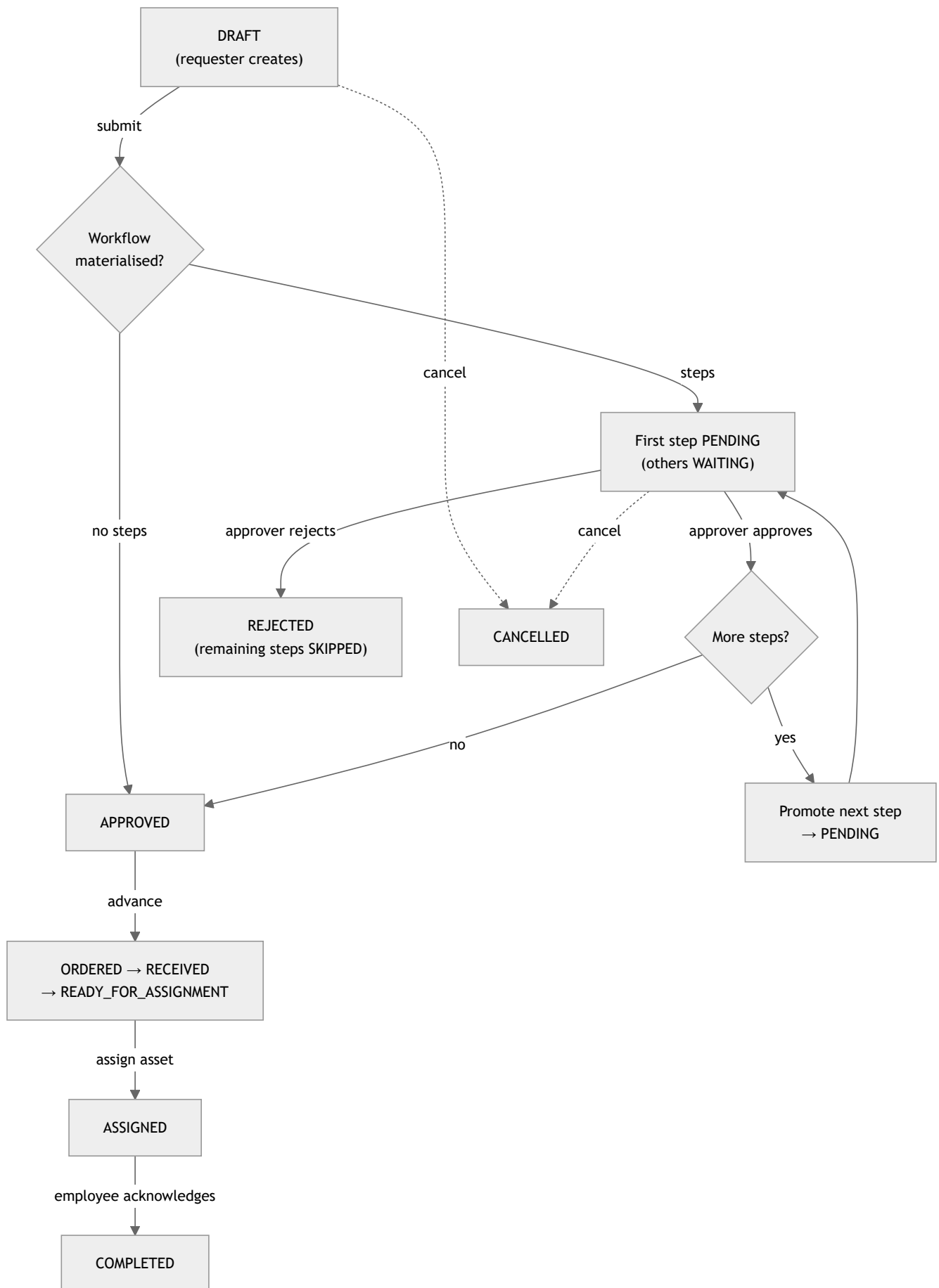
6.6 Auth & Password Fields

Password: 12–200 chars, must contain lowercase, uppercase, and a digit. Email: trimmed, lowercased, valid, ≤254. MFA code: **/^d{6}\$**/. Reset/verify tokens: ≥16 chars.

FIELD-DOC GAPS (1) **No create/invite-user form** exists on web or as an endpoint — users are created implicitly via asset import; the People screen only edits roles/status. (2) **No create-maintenance web form**. (3) The request-item **categoryId** is in the schema/defaults but has no UI input. These are the first candidates for the redesign's "missing forms" work (\$16).

7 Complete End-to-End Workflow

The flagship process is the **asset request**: an employee asks for equipment, it flows through a configurable approval chain, and once approved it is fulfilled (ordered/received/assigned) and the employee confirms receipt. The approval chain is materialised at submit time so a later configuration change cannot rewrite an in-flight request.



7.1 Lifecycle Steps

#	Step	Who	Data entered	System actions	Result status
1	Create draft	requests:create (on-behalf needs create-on-behalf)	type, priority, items, business reason, required-by	Generates REQ-YYYY-000000 ; denormalises manager/office/department from beneficiary; computes estimated cost	DRAFT
2	Submit	requester or an approver	—	Materialises applicable workflow steps given cost; step 0 PENDING, rest WAITING; if zero steps → jumps to APPROVED	first review status (e.g. MANAGER_APPROVAL_PENDING)
3	Approve step	requests:approve AND the matched approver of the current step	optional comment	Current step → APPROVED; next WAITING step → PENDING; notifies next approver or requester	next step status / APPROVED
3'	Reject step	same	comment (reason)	Step → REJECTED; all remaining steps → SKIPPED; notifies requester	REJECTED (terminal)
4	Fulfil	requests:approve	target status	Validated against the request state machine; fires ASSET_READY on READY_FOR_ASSIGNMENT	ORDERED → RECEIVED → READY_FOR_ASSIGNMENT → ASSIGNED → COMPLETED
5	Cancel	requester or approver	reason	Remaining approvals → SKIPPED. Legal from any non-terminal status except ASSIGNED	CANCELLED

WORKFLOW GAPS The fulfilment step (**advance**) writes **no audit entry** and fires notifications only for READY_FOR_ASSIGNMENT (ORDERED/RECEIVED/ASSIGNED/COMPLETED are silent). Recommended fixes in §16.

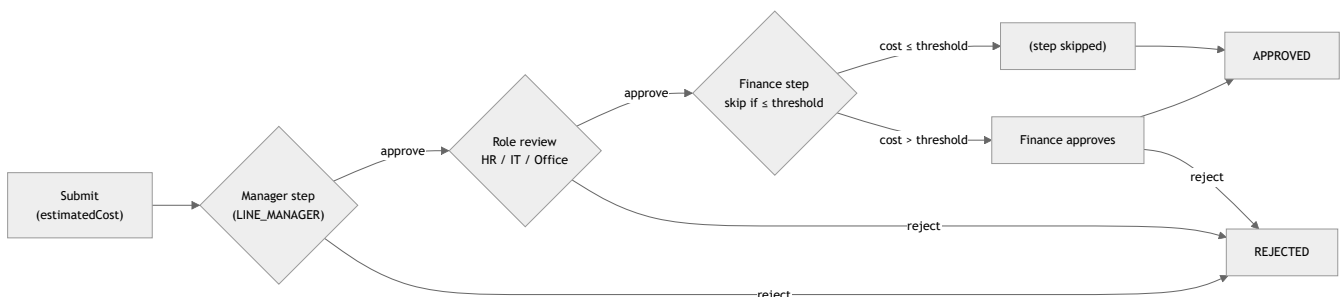
8 Approval Workflow

8.1 Routing Logic

Exactly one approval is **PENDING** per request at a time; later steps sit in **WAITING** so future approvers don't see the request prematurely. The current step's approver is resolved by **approver type**:

Approver type	Who may decide
USER	The specific named user (<code>approverUserId</code>).
LINE_MANAGER	The requester's <i>actual</i> manager (<code>profile.managerId</code>) — not any manager-role holder.
DEPARTMENT_HEAD	The requester's department head. GAP currently un-approvable — the head id is always passed as <code>null</code> .
ROLE (default)	Any holder of the step's role key (HR / IT_ADMIN / OFFICE_ADMIN / FINANCE / MANAGER).

Holding `requests:approve` is necessary but **not sufficient** — the actor must also match this step's approver, or the chain would be decorative. A step with a **cost threshold** is skipped when the estimate is $\leq$ threshold; an **unknown/absent cost never skips** (fail-safe — it goes to a human).



8.2 Seeded Workflow Definitions

Workflow definitions and steps are Super-Admin-editable config; the role→status mapping and skip logic are pure code. The **it-equipment** definition (`requestType = null`) is the catch-all fallback.

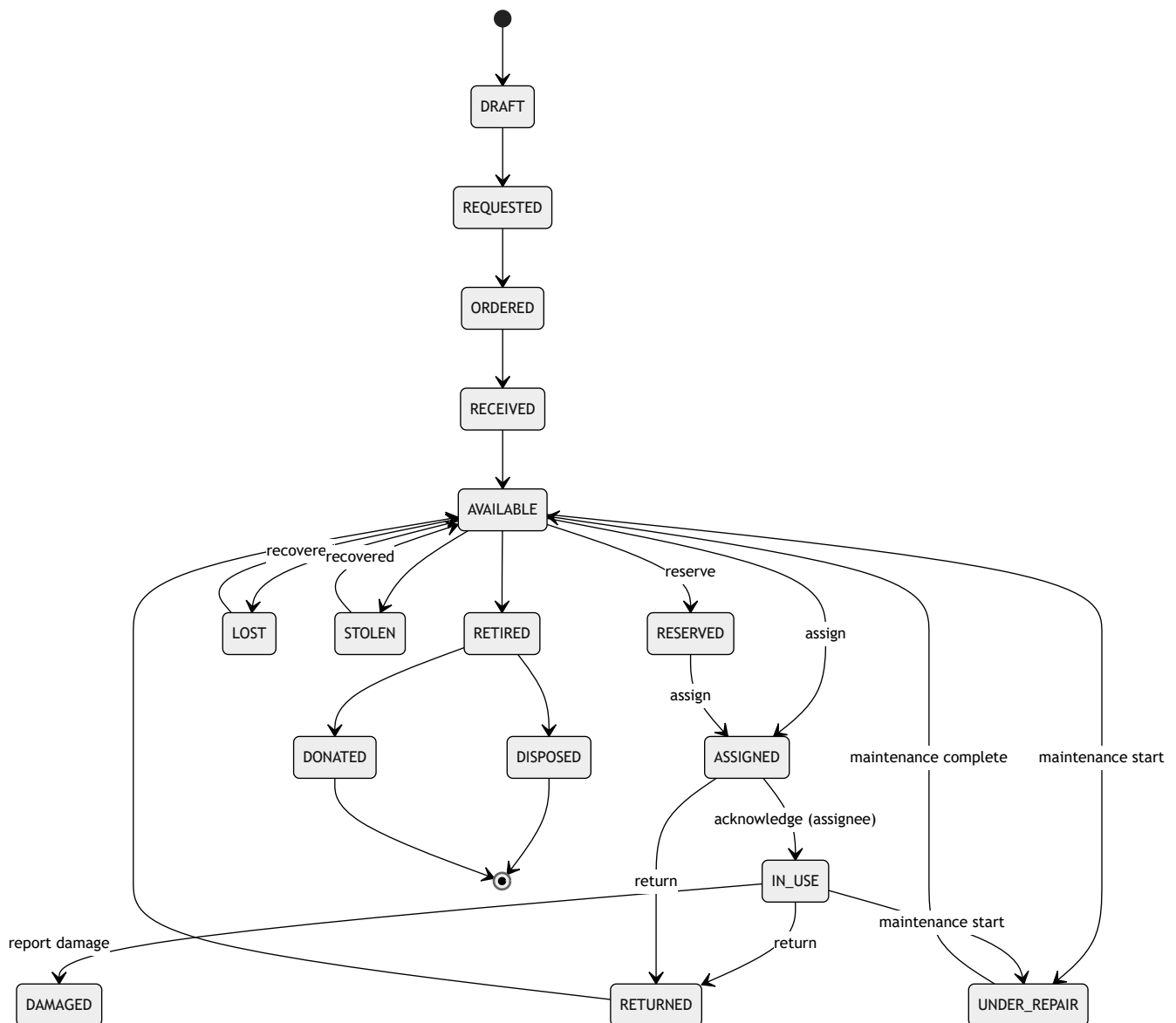
Key	Applies to	Steps (order · approver · SLA · threshold)
it-equipment (catch-all)	any type without a specific definition	1 Manager (LINE_MANAGER, 48h) · 2 HR (48h) · 3 IT (72h) · 4 Finance (72h, skip $\leq$ \$250)
onboarding-equipment	NEW_EMPLOYEE_ONBOARDING	1 HR (24h) · 2 IT (48h) · 3 Finance (skip $\leq$ \$1000)
furniture	OFFICE_REQUIREMENT	1 Manager (48h) · 2 Office Admin (72h) · 3 Finance (skip $\leq$ \$500)
kitchen	KITCHEN_REQUIREMENT	1 Office Admin (48h) · 2 Finance (skip $\leq$ \$200)
repair	REPAIR	1 IT review (24h)

SLA GAP Each approval stores an `slaDueAt` (now + SLA hours) but **no escalation job acts on it** — overdue approvals are not escalated or reassigned. A strong redesign candidate (\$16).

9 Status Transition Documentation

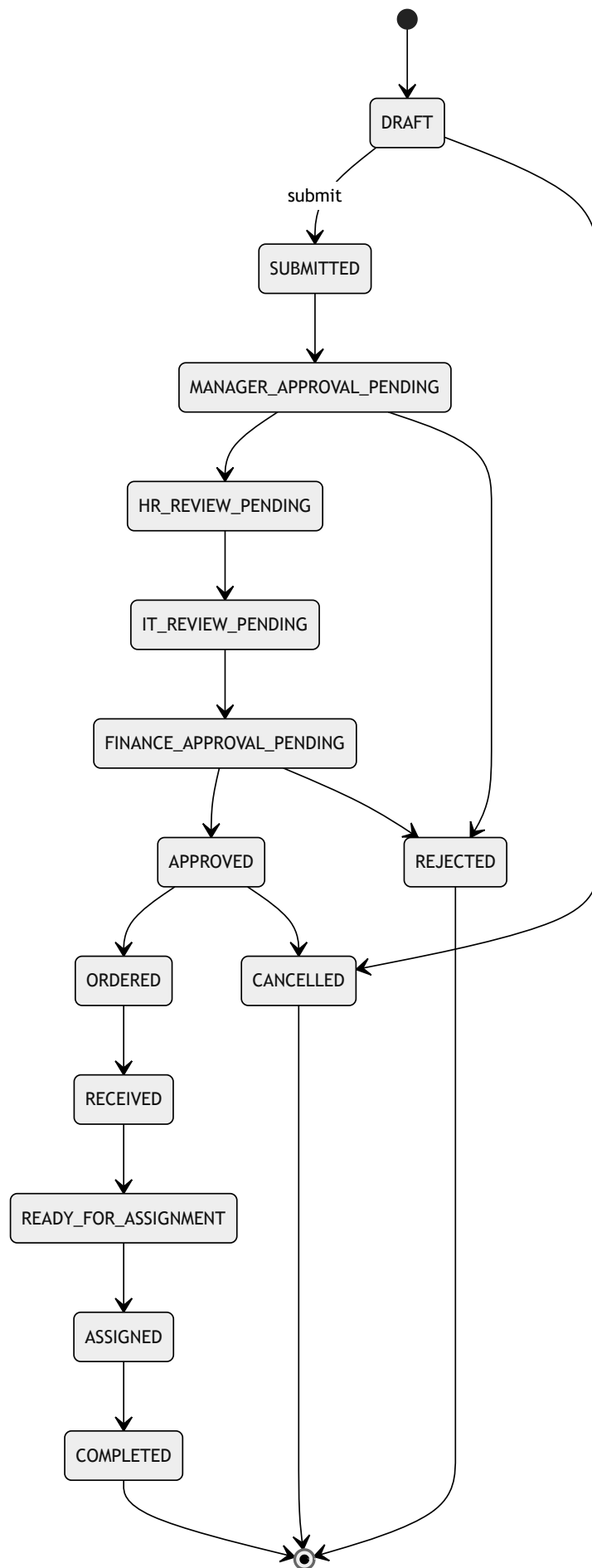
Every entity transition is guarded by a declarative state machine that rejects illegal moves (`IllegalTransitionError`). An identity transition (from = to) is always legal. Below: the state diagrams and the who/trigger/rules for each entity.

9.1 Asset (18 states; terminal: **DISPOSED, DONATED**)



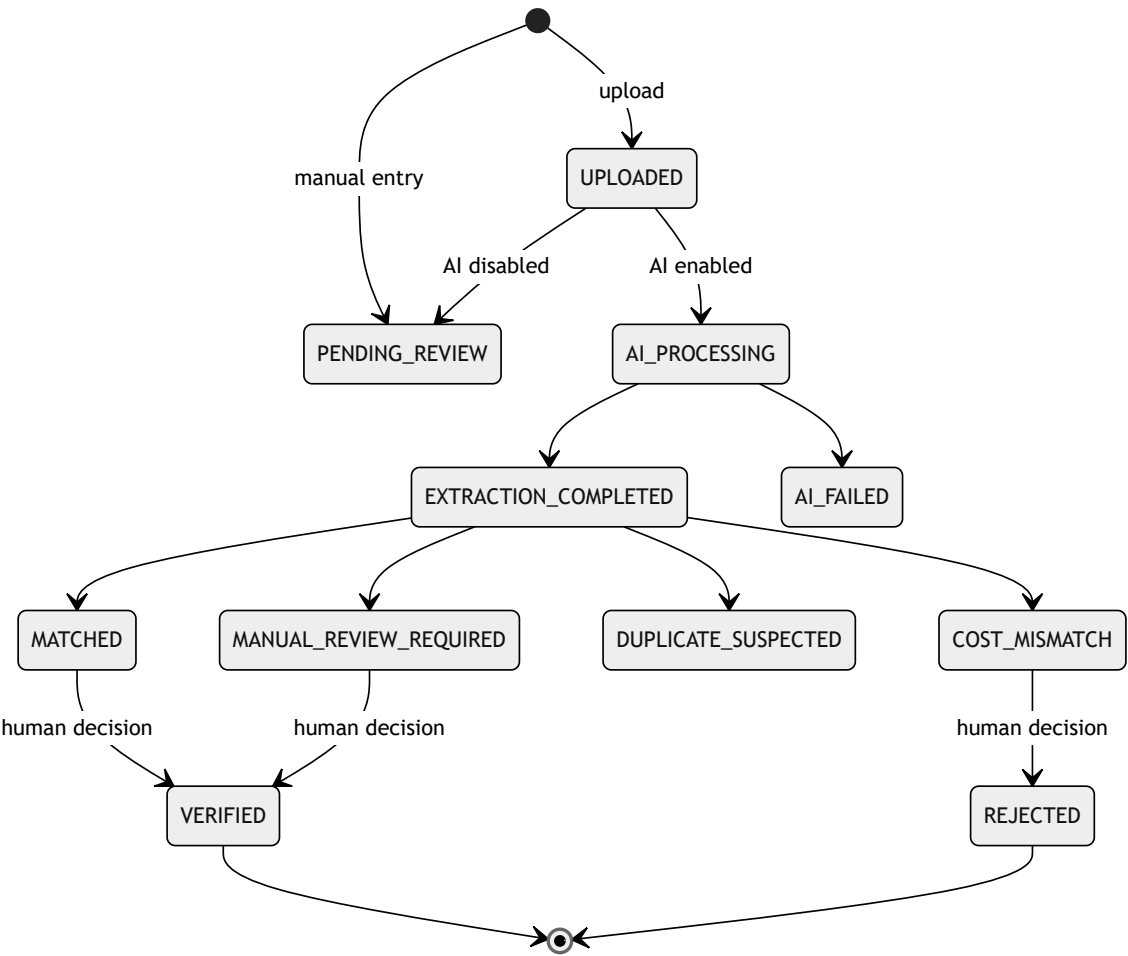
Also present: IN_STORAGE, IN_TRANSIT. Custody statuses = ASSIGNED/IN_USE; assignable = AVAILABLE/RESERVED. Every status change writes an `AssetConditionLog` and an audit row. Assignment requires `trackingType = INDIVIDUAL` ; only the assignee can acknowledge (→ IN_USE).

9.2 Request (16 states; terminal: **COMPLETED, REJECTED, CANCELLED**)



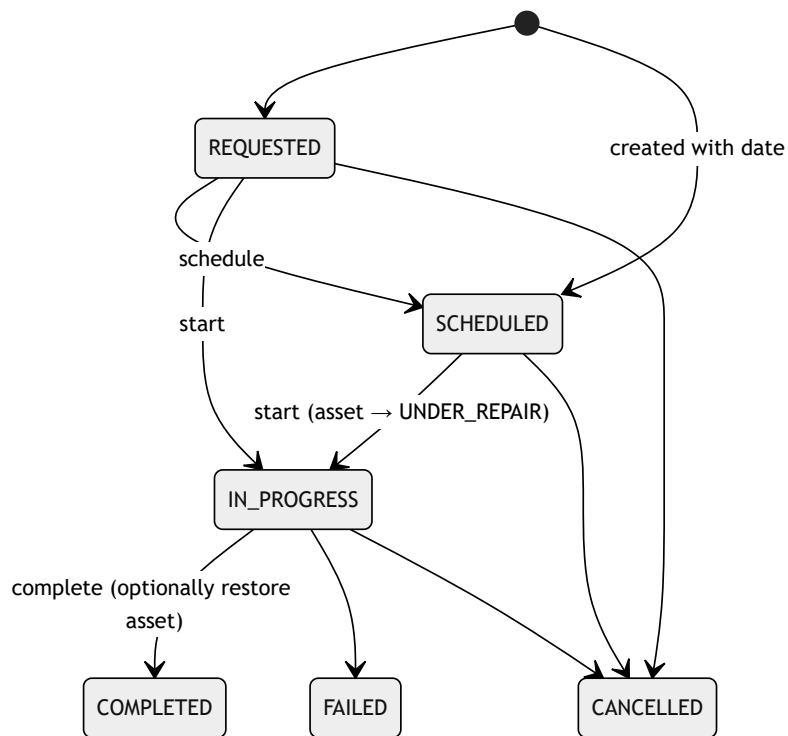
Review-step order lives in configuration (steps are mutually reachable in the machine). **OFFICE_ADMIN_REVIEW_PENDING** and **INVENTORY_RESERVED** also exist. **ASSIGNED** may only advance to COMPLETED — never CANCELLED (the employee already holds the asset).

9.3 Invoice Verification (terminal: VERIFIED, REJECTED — human only)



The deterministic engine derives a machine-observable outcome (precedence: DUPLICATE > COST_MISMATCH > QUANTITY_MISMATCH > SERIAL_MISMATCH > ASSET_MISSING > PARTIALLY_MATCHED > MANUAL_REVIEW > MATCHED) but **never** emits VERIFIED/REJECTED — those require an authenticated human (**assertHumanDecisionOnly**). Payment status (UNPAID/PARTIALLY_PAID/PAID/OVERDUE/DISPUTED/CANCELLED) is a stored field, not a state machine.

9.4 Maintenance (terminal: COMPLETED, CANCELLED, FAILED)



9.5 User Account (INVITED, ACTIVE, SUSPENDED, DEACTIVATED)

No formal state machine. `setStatus` (users:manage) moves between ACTIVE ↔ SUSPENDED/DEACTIVATED — cannot change your own status, and the last active Super Admin cannot leave ACTIVE. Offboarding completion deactivates the leaver (unless self or last Super Admin).

10 Notification Flow

A notification always writes an in-app row synchronously, then queues email and push through the queue provider, respecting the catalogue's allowed channels and the user's preference. Mandatory types ignore preferences. Absent preference = opted-in.

Event	Recipient(s)	Channels	Mandatory
Approval required (submit / next step)	current-step approver (named user, requester's manager, or role holders)	in-app, email, push	Yes
Request approved (final)	requester	in-app, email, push	No
Request rejected	requester	in-app, email, push	Yes
Ready for assignment	beneficiary (else requester)	in-app, email	No
Return required (offboarding start)	the leaving user	in-app, email, push	Yes
Maintenance scheduled / due	requestedBy	in-app	No
Warranty expiring (sweep)	assignee (else creator)	in-app, email	No

NOTIFICATION GAPS The catalogue defines ~13 notification types that are never fired in code, e.g. NEW_REQUEST, ASSET_ASSIGNED, RECEIPT_CONFIRMATION, RETURN_OVERDUE, DAMAGE_REPORTED, INVOICE_UPLOADED (in-app), AI_PROCESSING_COMPLETED/FAILED, INVOICE_MISMATCH, LOW_STOCK, REPLACEMENT_DUE, SECURITY_ALERT. Teams/Slack channels exist in the enum but no catalogue entry uses them. Wiring these is high-value, low-cost (\$16).

11 Business Rules

The rules below are enforced in the domain/service layers (not merely UI) and are the invariants any redesign must preserve.

11.1 Money & Invoice Verification

- Money is `Decimal(14,2)`, never float; quantity is `Decimal(14,3)` so consumables issue fractionally. Arithmetic stays in decimal.js end-to-end.
- **Negatives rejected:** subtotal, discount, tax, shipping, quantity, unit price. Discount may not exceed subtotal.
- Invoice total is fixed: `subtotal - discount + tax + shipping + otherCharges`.
- Verification is a **pure, deterministic engine** (no DB, no network, no AI): line totals, subtotal/total, tax/discount validity, duplicates, serials, quantity/price, PO reconciliation. Rounding tolerance = 1 minor unit; larger deltas → `COST_MISMATCH`.
- The engine never returns VERIFIED/REJECTED — a human decides.

11.2 Duplicate Detection

- Invoice number unique per company; file dedup by SHA-256; serials duplicate-within-file = ERROR, clash-with-existing-asset = WARNING; asset serial unique per company (documented-exception override).

11.3 AI Governance

- Every provider call passes `resolveAiGate` first; global disable/pause short-circuits to "no document sent to an external provider".
- AI may **never** make final financial approvals (`mayAutoApproveFinancials()` returns false unconditionally — no config can enable it).
- Human review required unless a feature is explicitly AUTOMATIC and the company hasn't globally mandated review. Modes: DISABLED / SUGGESTION_ONLY / MANUAL_REVIEW_REQUIRED / AUTOMATIC_PROCESSING / RESTRICTED_TO_ROLES. Override precedence: office+role → office → role → company.
- AI-normalised text never overwrites the original; original extraction, corrections, and decision are separate rows.

11.4 Immutability & History

- No hard deletes for financial, assignment, or audit rows; soft-delete (`deletedAt`) only where archival is permitted.
- Transfers never overwrite — each move is a new row recording both sides; disposal is a recorded event, not a delete.
- Optimistic concurrency via a `version` integer on contended rows (stale → `CONCURRENT_MODIFICATION` 409).
- Asset price is write-once; only a Super Admin may correct it, always audited.

11.5 Security Controls

- Brute-force lockout (5 attempts → 15 min) on top of per-IP throttling; a locked account is refused even with correct credentials.
- Refresh tokens rotate with family-wide reuse detection; only hashes stored; MFA secret AES-256-GCM encrypted, never returned.
- Authority always re-derived from DB state — the frontend is never trusted for roles/permissions.
- QR encodes an opaque ULID, never asset data; resolution requires auth and honours scope.
- Malware scan is a documented hook — status defaults SKIPPED, not silently CLEAN. Documents served only via signed URLs.

11.6 Audit

- Append-only (no update/delete path). 33 action types incl. `ROLE_CHANGED`, `PERMISSION_CHANGED`, `ASSET_COST_CHANGED`, `DATA_EXPORTED`, `LOGIN_FAILED`. Diffs whitelist fields so secrets never enter the log; a failing audit write is logged loudly but never blocks the user.

12 Database Overview

PostgreSQL via Prisma 6 (~2,100-line schema, ~44 models, 7 migrations). Multi-tenant: a Prisma client extension injects `companyId` on every query. Financial/assignment/audit rows have no delete path; contended rows carry a `version` for optimistic locking.

12.1 Entities by Domain

Domain	Key models	Notable relations / notes
Identity & RBAC	Company, User, UserProfile, Role, Permission, RolePermission, UserRole, RefreshToken, VerificationToken, DeviceToken	User→Company; User↔Role (UserRole); Role↔Permission (RolePermission); refresh tokens hashed + family-linked for reuse detection.
Org structure	Office, Building, Floor, Room, Department, Vendor, PurchaseOrder	Office→Building→Floor→Room hierarchy; assets/requests reference office+department.
Catalogue	Category, Subcategory	Category→Subcategory; drives tracking type + asset classification.
Assets	Asset, AssetAssignment, AssetReturn, AssetTransfer, AssetConditionLog	Asset→Category/Subcategory/Office/Room/Department/Vendor/assignedUser; every status change logs a condition row; transfers/returns are immutable events.
Requests & workflow	AssetRequest, RequestItem, RequestApproval, RequestComment, WorkflowDefinition, WorkflowStep	Request→Items/Approvals/Comments; approvals reference approverRole or approverUser; at most one PENDING approval; workflow config is per-company editable.
Invoices	Invoice, InvoiceLine, InvoiceLineAssetLink, InvoiceDocument	Invoice→Vendor/Lines/Documents; lines link to Asset or InventoryItem; documents carry sha256 + scan status; extraction/correction/decision retained separately.
Maintenance	MaintenanceRecord	→Asset/Vendor; drives asset UNDER_REPAIR cascade; captures service cost + downtime.
Inventory	InventoryItem, InventoryMovement	Quantity-tracked stock + movement ledger; mobile stocktake syncs movements.
Lifecycle	OnboardingTemplate, OnboardingTemplateItem, OnboardingTask	Offboarding cannot complete while assets remain in custody unless a documented exception is supplied.
Notifications	Notification, NotificationPreference, DeviceToken	Per-user preferences; mandatory types ignore them; push prunes dead Expo tokens.
Audit & AI	AuditLog, AIConfiguration, AIUsage, ScheduledReport	Append-only audit; AI config per company with per-feature modes; AIUsage records provider cost/tokens; scheduled reports carry a cron + recipients.

12.2 Enum Catalogue (selected)

Enum	Values
AssetStatus (18)	DRAFT, REQUESTED, ORDERED, RECEIVED, AVAILABLE, RESERVED, ASSIGNED, IN_USE, IN_STORAGE, IN_TRANSIT, UNDER_REPAIR, DAMAGED, LOST, STOLEN, RETURNED, RETIRED, DISPOSED, DONATED
AssetCondition (6)	NEW, GOOD, FAIR, POOR, DAMAGED, UNUSABLE
TrackingType	INDIVIDUAL, QUANTITY
RequestStatus (16)	DRAFT, SUBMITTED, MANAGER/HR/IT/OFFICE_ADMIN_REVIEW/FINANCE_APPROVAL_PENDING, APPROVED, INVENTORY_RESERVED, ORDERED, RECEIVED, READY_FOR_ASSIGNMENT, ASSIGNED, COMPLETED, REJECTED, CANCELLED
RequestType (13)	NEW_EMPLOYEE_ONBOARDING, REPLACEMENT, DAMAGE, LOSS, UPGRADE, TEMPORARY_ASSIGNMENT, PROJECT_REQUIREMENT, OFFICE_REQUIREMENT, KITCHEN_REQUIREMENT, ACCESSIBILITY_REQUIREMENT, ADDITIONAL_EQUIPMENT, REPAIR, RETURN
ApproverType / ApprovalDecision	ROLE, USER, LINE_MANAGER, DEPARTMENT_HEAD / WAITING, PENDING, APPROVED, REJECTED, SKIPPED, DELEGATED
Invoice payment / verification	UNPAID, PARTIALLY_PAID, PAID, OVERDUE, DISPUTED, CANCELLED / UPLOADED, AI_PROCESSING, EXTRACTION_COMPLETED, MATCHED, COST_MISMATCH, DUPLICATE_SUSPECTED, MANUAL_REVIEW_REQUIRED, VERIFIED, REJECTED (+ more)
MaintenanceStatus / Type	REQUESTED, SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED, FAILED / SCHEDULED, REPAIR, INSPECTION, WARRANTY_CLAIM, CALIBRATION, CLEANING

UserStatus / AuditAction	INVITED, ACTIVE, SUSPENDED, DEACTIVATED / 33 action types (auth, asset, assignment, invoice, request, role/permission, export, disposal...)
-----------------------------	---

ASSUMPTION Entity counts and relations were assembled from the authoritative schema plus cross-references across the analysis. A full column-by-column ER diagram is recommended as a living artifact generated directly from `schema.prisma` (e.g. via `prisma-erd-generator`) rather than hand-maintained.

13 API Documentation

All routes carry the `/api/v1` prefix (health probes excepted). Auth is global-deny; the "Permission" column is the `@RequirePermissions` gate (blank = any authenticated user, with ownership enforced in the service). Interactive Swagger docs are served at `/api/docs` in non-production.

13.1 Authentication

Method	Path	Access	Purpose
POST	/auth/login	public, 10/min	Sign in; returns MFA-required or access token + refresh cookie
POST	/auth/refresh	public, 60/min	Rotate refresh → new access token
POST	/auth/logout	auth	Revoke refresh family; clear cookie
GET	/auth/me	auth	Current user: roles, permissions, scope
GET	/auth/sso/available · /auth/sso/entra · /auth/sso/entra/callback	public	Microsoft Entra OIDC sign-in
POST	/auth/forgot-password (5/min) · /auth/reset-password (10/min) · /auth/verify-email · /auth/change-password	public / auth	Password & email flows
POST	/auth/mfa/enrol · /auth/mfa/confirm · /auth/mfa/disable	auth	TOTP MFA lifecycle

13.2 Assets

Method	Path	Permission	Purpose
GET	/assets	assets:read	List (scoped; cost columns omitted without cost perm); rich filters
GET	/assets/export · /assets/by-qr/token · /assets/:id	assets:read	CSV export · QR resolve · detail
POST	/assets	assets:create	Create asset
POST	/assets/import	assets:import	Excel bulk import (≤15 MB)
PATCH	/assets/:id	assets:update	Update (optimistic version)
PATCH	/assets/:id/price	assets:cost:read	Record price (write-once; Super-Admin correction)
POST	/assets/:id/status · /assets/bulk/status	assets:update	Status change (state-machine validated) · bulk
POST	/assets/:id/assign · /assets/:id/return	assets:assign / :return	Assign / receive back
POST	/assets/assignments/:id/acknowledge	assignee only	Confirm receipt (→ IN_USE)

13.3 Requests

Method	Path	Permission	Purpose
GET	/requests (+ ?awaitingMe) · /requests/export · /requests/types · /requests/:id	requests:read	List / approvals inbox / types / detail
POST	/requests · /requests/:id/submit	requests:create	Create draft · submit (materialise chain)
POST	/requests/:id/decision	requests:approve + current-step approver	Approve/reject current step
POST	/requests/:id/advance	requests:approve	Fulfilment transitions
POST	/requests/:id/cancel · /requests/:id/comments	requests:cancel / :read	Cancel · comment

13.4 Invoices, Maintenance, People, Lifecycle

Method	Path	Permission	Purpose
--------	------	------------	---------

GET/POST	/invoices · /invoices/:id · /invoices/upload · /invoices/:id/reverify · /invoices/:id/lines/link · /invoices/:id/decision	invoices:read/upload/correct- extraction/verify	List/detail, upload, re-verify, link line, human decision
GET/POST	/maintenance (+ :id) · :id/schedule · :id/start · :id/complete · :id/cancel · assets/:id/repair-advice	maintenance:read/manage (advice + assets:cost:read)	Maintenance lifecycle + repair- vs-replace
GET/PATCH	/users · /users/export · /users/:id · /users/:id/roles · /users/:id/status	employees:read; roles:manage; users:manage	Directory + role/status management
GET/POST	/lifecycle/tasks (+ :id) · /lifecycle/onboarding · /lifecycle/offboarding (+ :id/complete)	employees:read; onboarding/offboarding:manage	On/offboarding task plans (asset-recovery gated)

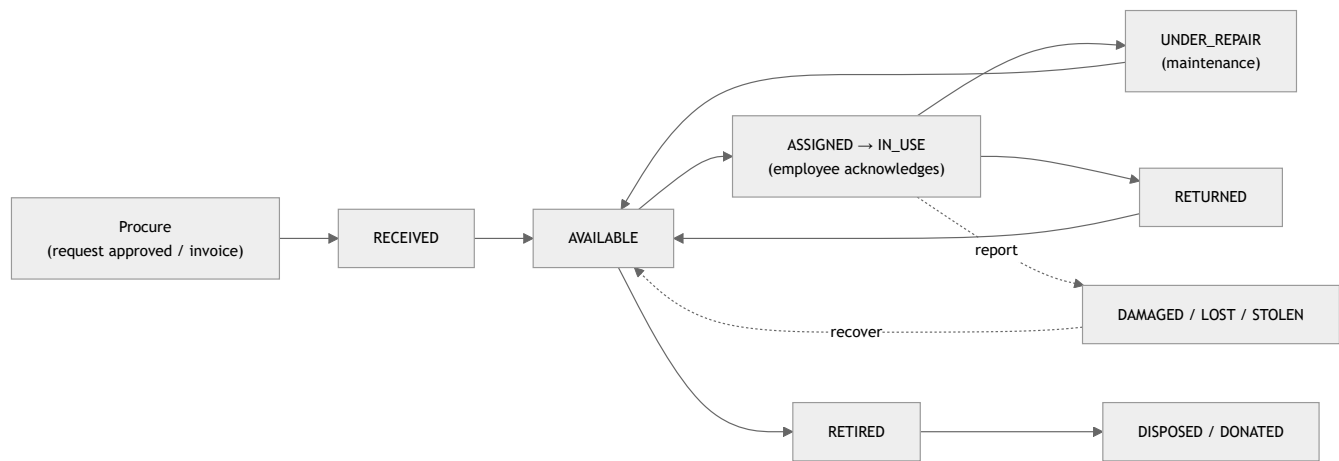
13.5 Reporting, Governance, Platform

Method	Path	Permission	Purpose
GET	/reports?type&format	reports:read (+ export; financial needs assets:cost:read)	7 report types, JSON/CSV/XLSX
GET	/audit	audit:read	Append-only trail, filterable
GET/PATCH	/ai-config	ai:configure	Read/update AI config
GET/POST/DELETE	/scheduled/reports (+ :id) · /scheduled/alerts/run	reports:read/export; settings:manage	Scheduled reports; trigger alert sweep
GET	/offices · /departments · /categories · /vendors	auth (vendors: vendors:read)	Reference data
GET/POST/PATCH	/notifications (+ unread-count, preferences, :id/read, read-all)	auth (self-scoped)	In-app centre + preferences
POST/GET	/mobile/devices · /mobile/assets/delta · /mobile/inventory/sessions · /mobile/sync	auth; assets:read; inventory:read/adjust	Device tokens, delta sync, offline stocktake
GET	/storage/preview/:doc · /storage/:key	invoices:read; signed HMAC URL	Inline preview · signed download
GET	/health/live · /health/ready	public	Liveness · readiness (DB critical, Redis non- critical, mock providers report "mocked")

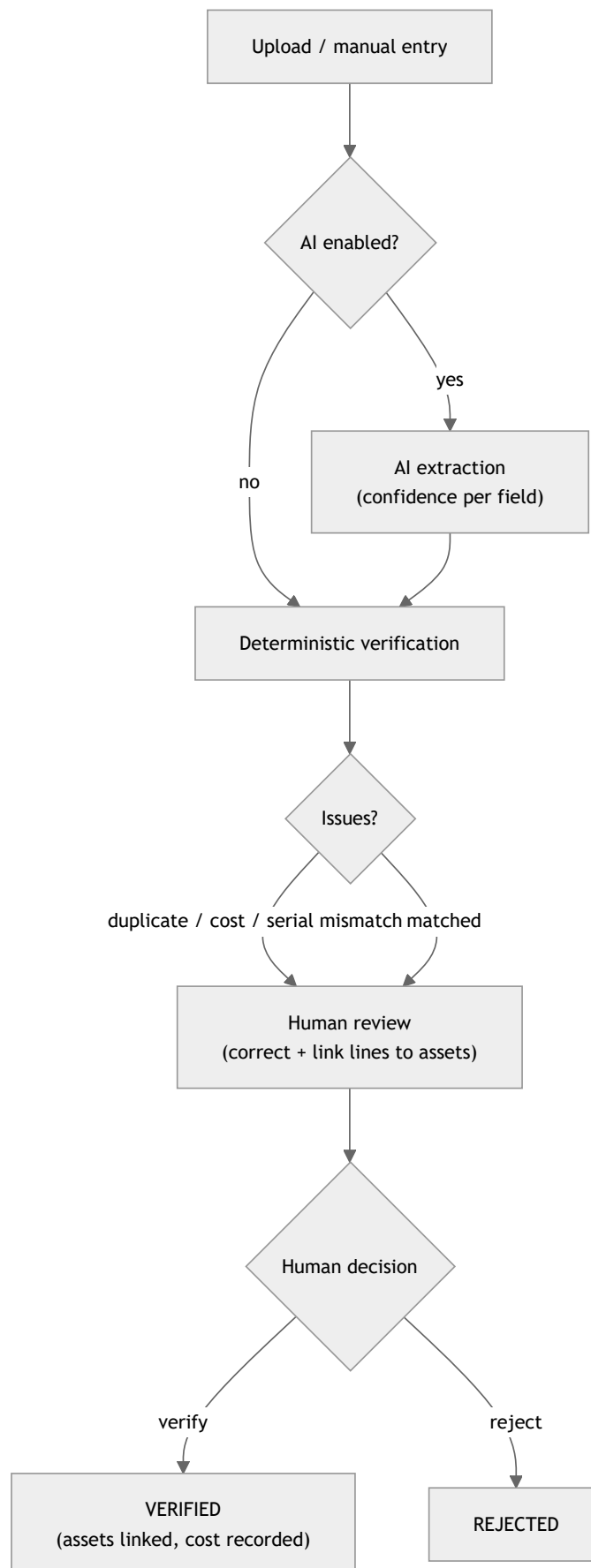
14 Workflow Diagrams & Flowcharts

Diagrams are placed with their sections (architecture §2.1, role interaction §3.3, request lifecycle §7, approval routing §8.1, state machines §9). Two consolidated end-to-end journeys follow.

14.1 Full Asset Journey (procure → dispose)



14.2 Invoice → Asset Reconciliation



15 UX / UI Improvement Suggestions

The application is already coherent (shared tokens, consistent status pills, empty/loading/error states, permission-aware navigation). The following raise it to enterprise polish and reduce friction.

Area	Issue observed	Recommendation	Priority
Dead links	<code>/my-requests</code> , <code>/settings/notifications appearance security</code> , <code>/help</code> are linked but unimplemented.	Implement the settings/help pages (notification & appearance preferences already have backing endpoints) or remove the links.	High
Silent validation	Disabled submit buttons with no explanation (e.g. request submit needed reason ≥ 10 chars). One instance already fixed on mobile.	Keep primary actions tappable; validate on press with a precise inline message; always confirm success and surface API errors. Apply consistently across all forms.	High
Click reduction	Create request is draft-then-submit (2 calls); asset assignment and status changes are multi-screen.	Add inline quick-actions (assign/return/status from the list row via a menu); one-tap "raise standard request" templates.	Medium
Search & filters	Global asset search only; list filters are basic (status/type); no saved views.	Add multi-facet filters (category, office, vendor, warranty window, assignee), saved filter presets, and column sorting on all lists.	Medium
Bulk actions	Only bulk asset status change exists.	Extend bulk operations to assignment, transfer, QR-label print, export-selected, and request approvals (approve many).	Medium
Mobile parity	Invoice verify, maintenance manage, people management are web-only.	Add mobile approve-invoice and start/complete-maintenance for field managers; keep heavy config on web.	Medium
Feedback & activity	Notifications are in-app on web, push-only on mobile; no per-record activity timeline.	Add a unified activity timeline (audit + comments + status changes) on asset/request detail; a mobile in-app notification centre.	Medium
Consistency	Some forms use react-hook-form + Zod; others (invoice upload, maintenance complete, user manage) use ad-hoc <code>useState</code> .	Standardise every form on react-hook-form + the shared Zod contract; one Field/Form component set.	High
Accessibility	Baseline present (WCAG-AA palette, focus states).	Formal WCAG 2.1 AA audit: keyboard traps, ARIA on custom selects, table semantics, live-region announcements for async results.	Medium
Onboarding	No first-run guidance.	Add an empty-state "get started" checklist and contextual help; role-aware default landing.	Low

16 Missing Features & Enhancement Recommendations

Grouped by value. Items marked **GAP** are places where the backend already assumes a feature that isn't wired end-to-end — the cheapest, highest-value wins.

16.1 Close Existing Gaps (highest ROI)

Gap	What to do
SLA escalation GAP	<code>slaDueAt</code> is stored but never acted on. Add a scheduled job to escalate/reassign/notify overdue approvals; surface an "overdue" badge in the approvals inbox.
Unfired notifications GAP	Wire the ~13 catalogued-but-unfired types (asset assigned, receipt confirmation, return overdue, damage reported, invoice mismatch, low stock, security alert...). Enable Teams/Slack channels via the existing chat provider.
Fulfilment audit/notify GAP	<code>advance()</code> emits no audit and mostly no notifications. Add audit + status notifications for ORDERED/RECEIVED/ASSIGNED/COMPLETED.
DEPARTMENT_HEAD approver GAP	Currently un-approvable (head id passed as null). Resolve the department head and wire the branch, or remove the approver type.
Scheduled-report runner GAP	Scheduled reports store <code>nextRunAt</code> but nothing executes/delivers them. Add a runner (BullMQ repeatable job) that builds + emails on schedule.
Missing forms GAP	Build a proper create/invite-user flow (endpoint + form; contract partly exists) and a create-maintenance web form.
Runtime role management GAP	Expose create-role / grant-permission endpoints that call <code>assertGrantAllowed</code> ; add scope config for custom roles.

16.2 Automation & Productivity

- **Request & onboarding templates:** pre-defined equipment bundles ("new developer kit") that raise a multi-item request in one tap.
- **Recurring requests / scheduled procurement:** auto-raise for consumables and licence renewals.
- **Approval delegation & out-of-office:** temporary delegate for approvers; auto-route while away.
- **Bulk approvals** and **bulk assignment** from the list.
- **Auto-replacement suggestions** from depreciation/warranty data (the data exists; surface as an action).

16.3 Records, Search & Collaboration

- **Attachments & comments on more entities** (assets, maintenance) — invoices already have documents; extend the pattern.
- **Per-record activity timeline** (audit + comments + status) on detail screens.
- **Advanced/saved search** across all modules; global command palette.
- **Custom fields** per category (organisations track different attributes per asset type).

16.4 Analytics, Reporting & Integrations

- **Analytics dashboards** beyond the home KPIs: spend trends, utilisation, TCO, maintenance cost per asset, forecast.
- **Reporting enhancements:** scheduled delivery (see gap), report builder, PDF export, drill-through.
- **Integrations:** HRIS (auto-provision employees), accounting (invoice/PO sync), MDM/RMM (auto-discover devices), calendar (maintenance), Slack/Teams (approvals in-channel), webhooks/public API.
- **QR/barcode label printing at scale**, and asset check-in/out kiosks.

16.5 Enterprise & Trust

- **SSO/SCIM** for provisioning (Entra sign-in exists; add SCIM auto-provision + Google SSO).
- **Data retention & export/erasure** (GDPR), configurable per company.
- **Contract/warranty renewal management**, insurance and compliance registers.
- **Multi-currency** normalisation and per-office locale.

17 Development Best Practices

The codebase already follows strong patterns (typed contracts, provider abstractions, state machines, deny-by-default auth, redaction). The recommendations preserve those strengths while closing operational gaps.

Dimension	Current state	Recommendation
Architecture	Clean monorepo; pure <code>domain</code> package; provider abstractions; single response/error model.	Keep the domain package I/O-free. Document module boundaries; add architecture decision records (ADRs) for future changes.
Database	Prisma schema is the source of truth; multi-tenant extension; optimistic locking; immutable financial/audit rows.	Generate the ER diagram from <code>schema.prisma</code> in CI; add DB-level indexes review for the largest lists; consider partitioning audit by month at scale.
API	Versioned prefix, RFC 9457 errors, pagination contract, Swagger in dev.	Publish an OpenAPI spec artifact from the Zod contracts; add API response caching headers where safe; keep the envelope stable as a public contract.
Folder / components	Feature modules on API; App Router on web; shared ui-tokens.	Extract a shared web component library (Field/Form/Card/StatusPill/Table) — some forms still hand-roll <code>useState</code> . Mobile already has a design-system (<code>src/components/ui.tsx</code>); mirror it on web.
Coding standards	ESLint/Prettier bases in <code>packages/config</code> ; strict TS; Node-style <code>.js</code> specifiers.	Enforce via CI (see below). Add commit linting + conventional commits; codeowners for domain/security files.
Error handling	Single global filter; typed <code>AppError</code> ; Prisma error mapping; internal detail never leaked.	Maintain the error-code catalogue as a shared enum; add client-side error boundaries per route (web already has some).
Logging / observability	Redaction, request ids, correlation ids, structured logs.	Ship logs + metrics to a platform (OpenTelemetry traces, request/DB latency, queue depth); add health/uptime alerting.
Testing	Vitest unit + integration (real DB, swc for decorator metadata); <code>pnpm verify</code> gate.	Populate the empty <code>e2e</code> workspace (Playwright web + Detox/Maestro mobile happy-paths); raise coverage on services; contract tests from Zod schemas.
CI/CD GAP	No committed CI pipeline; the gate is local <code>pnpm verify</code> .	Add GitHub Actions: install → typecheck → lint → test (with an ephemeral Postgres) → build → (optional) deploy. Block merge on red; run migrations check.
Security	argon2id, MFA, rotating refresh, helmet/CSP, rate limits, audit, signed URLs, malware-scan hook.	Wire a real malware scanner; add dependency-scanning + SAST in CI; periodic secret rotation; pen-test before enterprise GA.
Scalability	Single-VPS Docker; in-process queue/cache defaults; Redis available.	Switch queue/cache/rate-limit to Redis for multi-instance; move the in-process alert timer to a BullMQ repeatable job; make the API horizontally scalable behind the proxy; object storage (S3/Azure) for documents in production.

18 Future Roadmap

A pragmatic sequencing that front-loads low-cost, high-trust wins (closing gaps) before feature expansion.

Phase	Theme	Highlights
Phase 1 — Harden & complete (weeks)	Close gaps, operationalise	SLA escalation job; wire unfired notifications + Teams/Slack; fulfilment audit/notify; scheduled-report runner; create-user & create-maintenance forms; DEPARTMENT_HEAD wiring; remove/implement dead links; standardise forms on RHF+Zod; add CI + first E2E happy-paths; Redis for queue/cache/rate-limit in prod; real malware scan.
Phase 2 — Automate & delight	Reduce clicks, add leverage	Request/onboarding templates; recurring requests; approval delegation/out-of-office; bulk approvals/assignment; per-record activity timeline; attachments/comments on assets & maintenance; saved filters + advanced search; mobile approve-invoice & maintenance actions.
Phase 3 — Insight & integrate	Analytics + ecosystem	Analytics dashboards (spend/utilisation/TCO/forecast); report builder + PDF; HRIS/accounting/MDM integrations; public API + webhooks; custom fields per category; QR label printing at scale.
Phase 4 — Enterprise & scale	Trust, compliance, multi-tenant scale	SSO/SCIM provisioning + Google SSO; GDPR retention/export/erasure; contract/warranty renewal & compliance registers; multi-currency; horizontal scaling + observability platform; SOC2-readiness.

19 Appendix & Glossary

19.1 Glossary

Term	Meaning
RBAC	Role-Based Access Control — permissions grouped into roles; guards check permission strings, not role names.
Data scope	Which records a user may see: ALL / DEPARTMENT / DIRECT_REPORTS / OWN. Orthogonal to permissions.
Write-once	A value settable once (asset price); afterwards only a Super Admin may correct it, always audited.
Materialise (a workflow)	Snapshot the applicable approval steps at submit time so later config changes can't rewrite an in-flight request.
Approver type	How a step's approver is resolved: USER / LINE_MANAGER / DEPARTMENT_HEAD / ROLE.
Deterministic verification	Pure, testable invoice checks (no AI, no DB) — AI may assist extraction but never decides.
Provider abstraction	Abstract class as DI token; concrete implementation chosen from an env var at boot (storage/AI/mail/queue/cache/push/SSO).
Response envelope	<code>{ data, meta }</code> wrapper on every success response; errors use RFC 9457 problem+json.
Custody / assignable	Custody statuses = ASSIGNED/IN_USE; assignable = AVAILABLE/RESERVED.
Optimistic locking	A <code>version</code> integer on contended rows; a stale version is rejected with 409.

19.2 Environment Contract (required secrets)

The API refuses to boot on an invalid environment. **Required (no default):** `DATABASE_URL`, `REDIS_URL`, `JWT_ACCESS_SECRET` (≥ 32), `JWT_REFRESH_SECRET` (≥ 32), `MFA_ENCRYPTION_KEY` (≥ 32). **Conditionally required** by provider selection: Azure/S3 storage creds, Azure/Anthropic AI creds, SMTP host, Expo token, Entra client credentials. Most operational settings have safe defaults (see §2.5).

19.3 Source-of-Truth Map

Concern	Authoritative source
Roles, permissions, scopes	<code>packages/domain/src/permissions.ts</code>
State machines & workflow rules	<code>packages/domain/src/*-status.ts</code> , <code>workflow.ts</code> , <code>state-machine.ts</code>
Money, verification, AI-gate, depreciation	<code>packages/domain/src/money.ts</code> , <code>invoice-verification.ts</code> , <code>ai-config.ts</code> , <code>depreciation.ts</code>
API payload schemas	<code>packages/contracts/src/*.ts</code> (Zod)
Database schema	<code>apps/api/prisma/schema.prisma</code> + migrations
Endpoints	<code>apps/api/src/**/*.controller.ts</code>
Web screens / nav	<code>apps/web/src/app/**</code> , <code>components/app-shell.tsx</code>
Mobile screens / nav	<code>apps/mobile/app/**</code> , <code>(tabs)/_layout.tsx</code>
Deployment	<code>docker-compose.vps.yml</code> , <code>docs/DEPLOY.md</code>

19.4 Known Gaps Summary

Consolidated from the analysis for quick tracking: dead web links (§5.2); no create-user/invite & no create-maintenance web forms (§6.6); SLA never escalated (§8.2); DEPARTMENT_HEAD un-approvable (§8.1); fulfilment (advance) no audit/notify (§7.1); ~13 unfired notification types + unused Teams/Slack (§10); scheduled-report runner missing (§16.1); runtime role management not implemented (§3.6); no CI, empty E2E workspace (§17); malware scan is a hook only (§11.5).

END OF DOCUMENT This SRS baselines TechpioAsset as-built. Sections 15–18 form the recommended agenda for the planned workflow redesign; the gaps in §19.4 are the fastest, highest-trust place to begin.